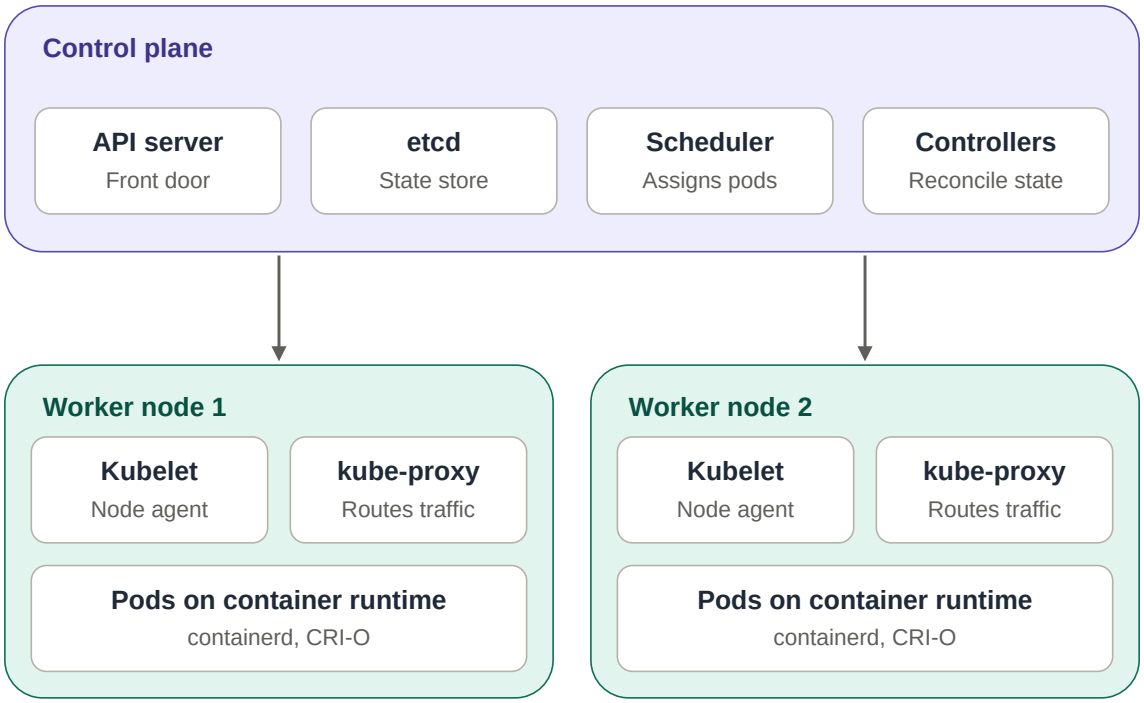
Part 1: Kubernetes Architecture

A Kubernetes cluster has two halves. The control plane makes decisions, and the worker nodes run your workloads.



Control plane components

- `kube-apiserver` is the only entry point. `kubectl`, the other components and the kubelets all talk to it through a REST API, and it handles authentication, authorization and validation.
- `etcd` is a consistent key-value store holding all cluster state. Only the API server reads and writes it directly, so backing up etcd means backing up the cluster.
- `kube-scheduler` watches for pods with no node assigned and picks one, based on resource requests, affinity rules, taints and available capacity.
- `kube-controller-manager` runs the control loops (Deployment, ReplicaSet, Node, Job and others). Each loop compares the desired state with the actual state and acts to close the gap.
- `cloud-controller-manager` is optional. It integrates with cloud load balancers, routes and node lifecycle.

Worker node components

- `kubelet` is the node agent. It receives pod specs from the API server, tells the container runtime to start them, runs probes and reports status back.
- `kube-proxy` maintains networking rules (iptables or IPVS) so that Service IPs route to the right pods.
- The container runtime (`containerd` or `CRI-O`) actually pulls images and runs containers, via the Container Runtime Interface (CRI).

Request flow: kubectl apply -f deployment.yaml

1. `kubectl` sends the manifest to the API server, which validates it and stores it in etcd.
2. The Deployment controller creates a ReplicaSet, which creates the pod objects.
3. The scheduler sees unassigned pods and writes a node name onto each one.
4. The kubelet on that node notices the assignment and asks the runtime to start the containers.
5. The kubelet reports status back, and the controllers keep reconciling if anything drifts, such as restarting a crashed pod.

Key design idea

Everything goes through the API server, and every component works by watching state and reconciling toward the desired state, not by being called directly.

Part 2: A Detailed Example

Deploying a web app and following it through the cluster. We will deploy a 3-replica nginx app, expose it, then trace what each component does.

Step 1: The manifests

```
# app.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web
spec:
  replicas: 3
  selector:
    matchLabels:
      app: web
  template:
    metadata:
      labels:
        app: web
    spec:
      containers:
      - name: nginx
        image: nginx:1.27
        ports:
        - containerPort: 80
        resources:
          requests:
            cpu: 100m
            memory: 128Mi
          limits:
            memory: 256Mi
        readinessProbe:
          httpGet:
            path: /
            port: 80
---
apiVersion: v1
kind: Service
metadata:
  name: web-svc
spec:
  selector:
    app: web
  ports:
  - port: 80
    targetPort: 80

kubectl apply -f app.yaml
```

Step 2: What happens, component by component

1. kubectl to API server

`kubectl` reads your kubeconfig, authenticates (certificate or token), and sends the manifests to `kube-apiserver`. The API server then runs three checks:

- Authentication: who are you?
- Authorization (RBAC): may you create Deployments in this namespace?

- Admission and validation: is the spec valid, and do any policies modify or reject it?

If all pass, it writes the Deployment and Service objects to **etcd**. At this point nothing is running yet. Only the desired state is recorded.

2. Deployment controller

The controller manager watches the API server and sees a new Deployment. It creates a **ReplicaSet** (`web-7d9f8c6b5`) with `replicas: 3`.

3. ReplicaSet controller

It sees a ReplicaSet wanting 3 pods but finding 0, so it creates 3 Pod objects (`web-7d9f8c6b5-abc12` and so on). These pods have **no node assigned** and sit in `Pending`.

4. Scheduler

The scheduler watches for pods with an empty `spec.nodeName`. For each pod it runs two phases:

- **Filtering** removes nodes that cannot host it (not enough free CPU or memory for the requests, taints the pod does not tolerate, node selectors that do not match).
- **Scoring** ranks the remaining nodes (spread across nodes, image already cached, balanced resource use).

It then writes the chosen node name into each pod via the API server. For example: pod 1 to `node-a`, pod 2 to `node-b`, pod 3 to `node-c`.

5. Kubelet (on each chosen node)

Each kubelet watches for pods assigned to its node. It then:

- asks the container runtime (via CRI) to pull `nginx:1.27`
- sets up the pod network through the CNI plugin, which gives the pod an IP such as `10.244.1.7`
- starts the container and runs the readiness probe
- reports status back to the API server (`Pending` then `Running`, `Ready: true` once the probe passes)

6. Endpoints and kube-proxy

The endpoints controller watches the Service's selector (`app: web`). When pods become Ready, their IPs are added to the Service's EndpointSlice. `kube-proxy` on every node sees that change and programs iptables or IPVS rules, so traffic to the Service's virtual IP (say `10.96.0.50:80`) is load-balanced across the three pod IPs.

Step 3: Verify it

```
kubectl get deploy,rs,pods -o wide
```

Example output:

```
NAME                  READY   UP-TO-DATE   AVAILABLE
deployment.apps/web   3/3     3            3

NAME                              DESIRED   CURRENT   READY
replicaset.apps/web-7d9f8c6b5     3         3         3

NAME                       READY   STATUS    IP           NODE
pod/web-7d9f8c6b5-abc12    1/1     Running   10.244.1.7   node-a
pod/web-7d9f8c6b5-def34    1/1     Running   10.244.2.4   node-b
pod/web-7d9f8c6b5-ghi56    1/1     Running   10.244.3.9   node-c
```

```
kubectl get svc web-svc
kubectl get endpointslices -l kubernetes.io/service-name=web-svc
kubectl describe pod web-7d9f8c6b5-abc12   # Events shows scheduler and kubelet steps
```

The `Events` output reads like the story above: `Scheduled`, then `Pulling`, `Pulled`, `Created`, `Started`.

Step 4: Self-healing in action

Scenario A: a pod is deleted

```
kubectl delete pod web-7d9f8c6b5-abc12
```

The ReplicaSet controller notices 2 pods exist but 3 are desired, and creates a replacement. The scheduler places it, the kubelet starts it, and the Service updates its endpoints. This usually takes a few seconds, and the Service keeps serving from the other two pods meanwhile.

Scenario B: a container crashes

The kubelet notices the container exited and restarts it according to `restartPolicy`. If it keeps failing, you will see `CrashLoopBackOff`, with growing delays between restarts. The pod object stays and the kubelet does the restarting, so no controller is involved at this level.

Scenario C: a whole node dies

1. The kubelet on `node-c` stops sending heartbeats.
2. After a grace period (roughly 40 seconds by default), the node controller marks the node `NotReady`.
3. Pods on it are marked for eviction after the toleration timeout (300 seconds by default for the not-ready taint).
4. The ReplicaSet controller creates replacement pods, and the scheduler places them on healthy nodes.

During all of this, `kube-proxy` stops routing to the dead pod because its readiness is gone from the EndpointSlice.

Scenario D: scaling

```
kubectl scale deploy web --replicas=5
```

Only the desired state changes in etcd. The ReplicaSet controller creates 2 more pods, and the same scheduling and kubelet flow follows.

Step 5: A rolling update

```
kubectl set image deploy/web nginx=nginx:1.28
kubectl rollout status deploy/web
```

1. The Deployment controller creates a **new ReplicaSet** for the new pod template.
2. It scales the new one up and the old one down gradually (by default at most 25% unavailable and 25% surge).
3. New pods must pass their readiness probe before old ones are removed, which is why the readiness probe matters for zero-downtime updates.
4. The old ReplicaSet is kept at 0 replicas so you can roll back:

```
kubectl rollout undo deploy/web
```

Step 6: Traffic path from the outside

A request to the app inside the cluster takes this path:

```
client pod
  -> web-svc (10.96.0.50:80)
  -> iptables/IPVS rule on the node
  -> one pod IP (10.244.x.x:80)
```

To reach it from outside the cluster, you would add a `Service` of type `LoadBalancer` or an `Ingress` (or Gateway API) in front of it, with the cloud controller or ingress controller doing the extra work.

What this example shows

- You only ever declare **desired state**. No component is told what to do directly.
- Every step is a **watch and reconcile** loop going through the API server, with etcd as the source of truth.
- The scheduler only decides placement, and the kubelet only runs what is assigned to its node.
- `requests` influence scheduling, `limits` are enforced at runtime, and `readinessProbe` controls whether traffic reaches the pod.